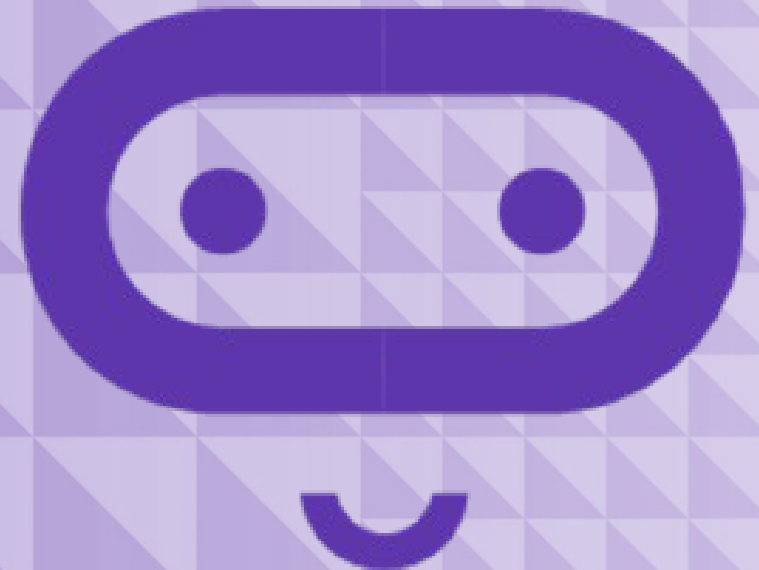


micro:bit

Unit 10: Radio communication

Lesson A

Understanding radio communication



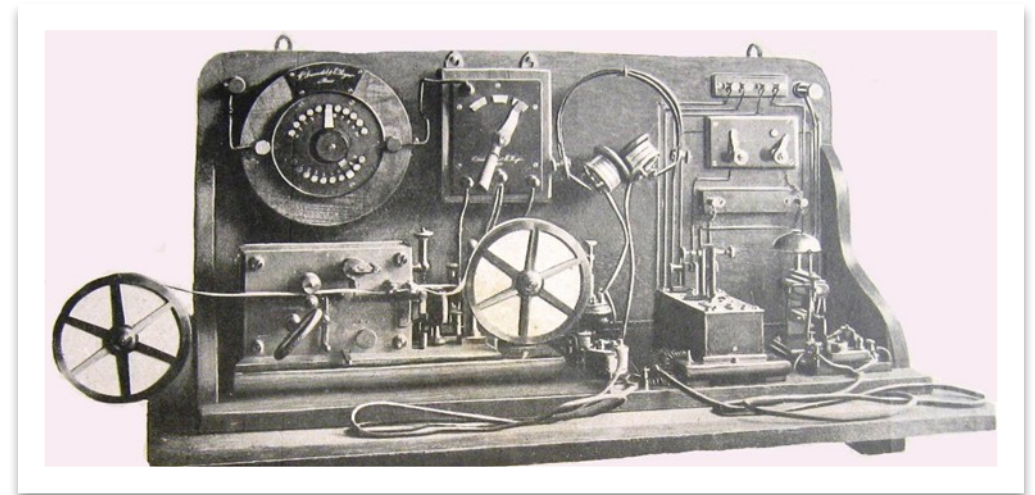
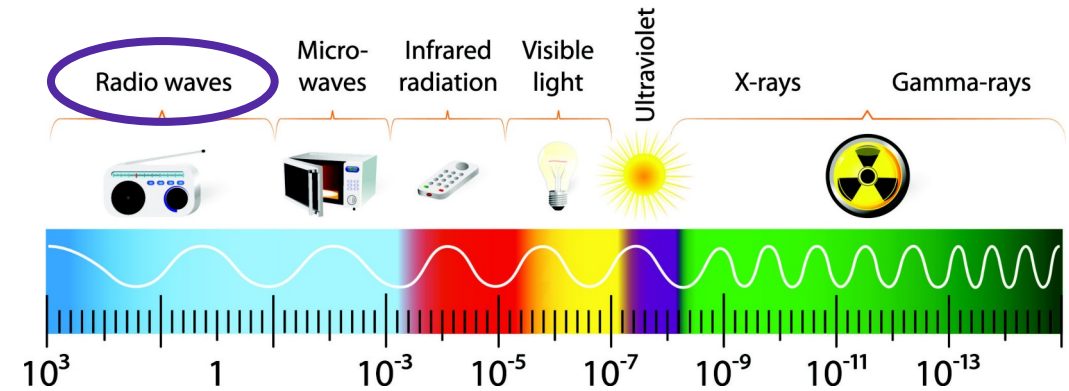
What we'll learn in Lesson A

- How radio communication works
- The radio functionality of the micro:bit

Radio communication

- Radios use **electromagnetic waves** to send messages through the air.
- A radio communication system requires a **transmitter** to send messages and a **receiver** to receive messages.
- The first radio was invented in 1894 by inventor Guglielmo Marconi in Italy and used **radio waves** to transmit and receive telegraph signals.

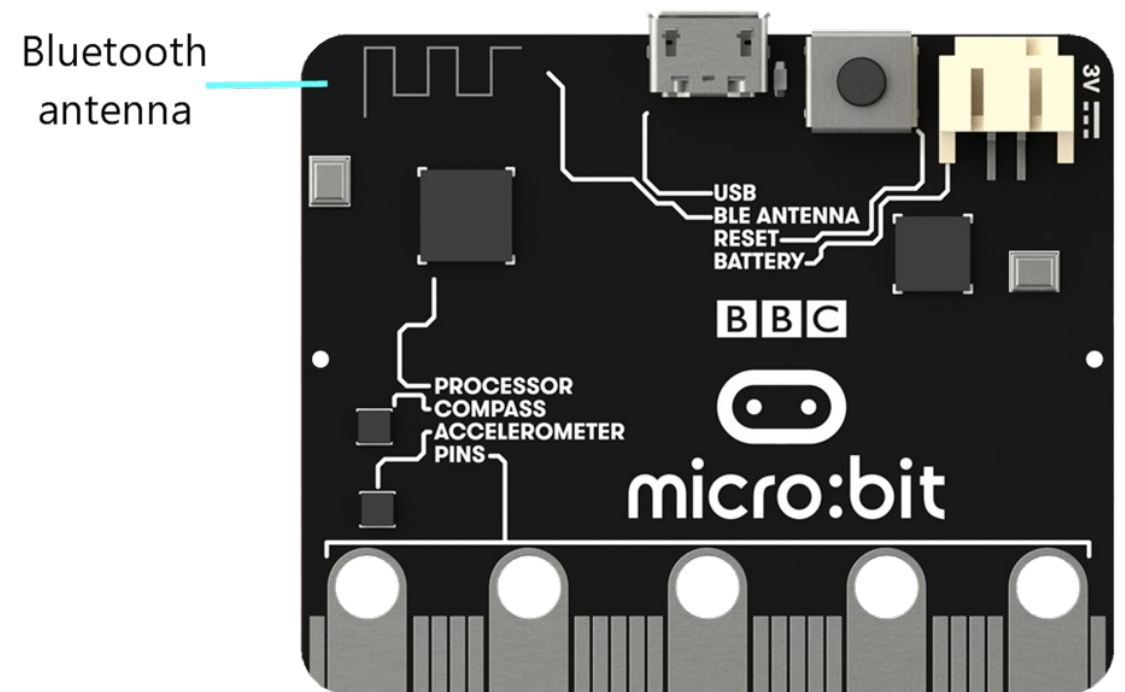
THE ELECTROMAGNETIC SPECTRUM



One of the first Radios

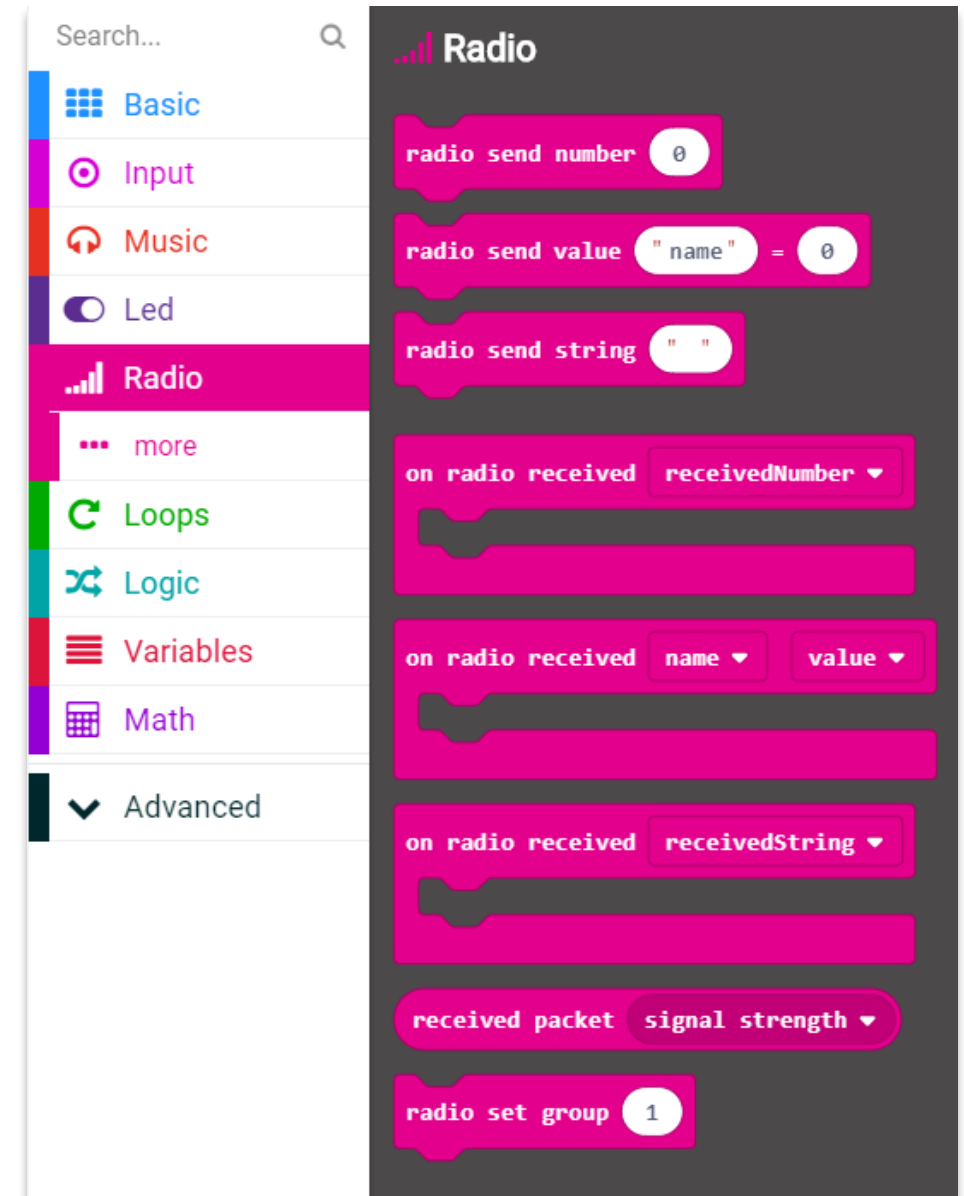
Radio on the micro:bit

On the back of the micro:bit, if you look closely in the upper left corner, there is an antenna, which is how the micro:bit sends and receives messages through Radio waves.



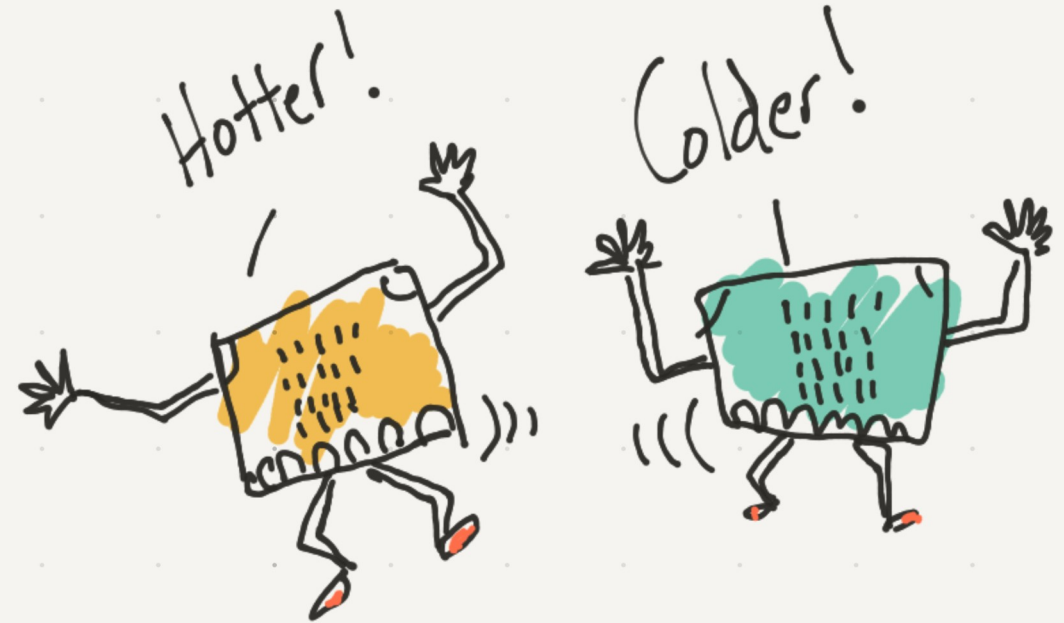
Programming with Radio

- The micro:bit allows you to communicate with other micro:bits in the area using the blocks in the **Radio category**.
- The micro:bit can act as a **Sender and/or Receiver** depending on what blocks you use in your program.
- You can send a **number**, a **string**, or a **string/number combination** in a message.
- You can give a micro:bit **instructions** on what to do when it receives a radio message.



Hot or Cold?

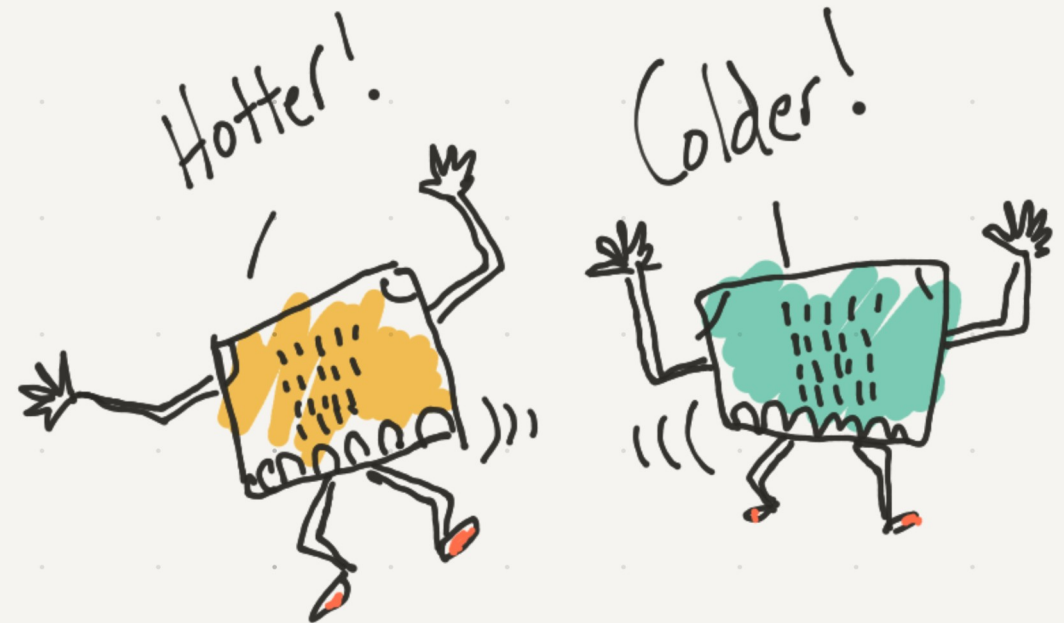
- One person picks an object in the classroom.
- Person A picks an object in the classroom.
- Person B tries to figure out what it is by taking one or more steps toward the object and then calling "Check".
- Person B responds with "Hotter" or "Colder" based on how close Person A is to the object.
- See how many Checks it takes. Then switch places!



Let's discuss!

Which strategy works better in this game:

- Taking just one step?
- Moving a greater distance each turn?
- How much variation was there in the Checker's responses?
- Did they respond with more than the standard responses?
- How helpful were those variations to the Finder?



What we learned in Lesson A

- How Radio communication works
- The radio functionality of the micro:bit



Next time: Lesson B

- Explore the radio toolbox
- Assess our learning with a quiz



Lesson B

Explore the Radio Toolbox



What we'll learn in Lesson B

- Programming using radio blocks to send and receive different types of data
- How much we've already learned

Marco Polo

- Who can explain the swimming pool game “Marco Polo”?
- We will be coding a form of this game using Radio communication on the micro:bit.
 - On button A pressed, the micro:bit will send the string “Marco”
 - On button B pressed, it will send the string “Polo”
 - Micro:bit will also display the strings received from the Radio



Search...

Basic

Input

Music

Led

Radio

Loops

Logic

Variables

Math

Extensions

Advanced

Your final code may look something like the following:

on start

radio set group 1

show leds

on button A pressed

radio send string "Marco"

play tone Low C for 1 beat

on button B pressed

radio send string "Polo"

play tone Middle C for 1 beat

on radio received receivedString

show string receivedString

Search...

Basic

Input

Music

Led

Radio

Loops

Logic

Variables

Math

Extensions

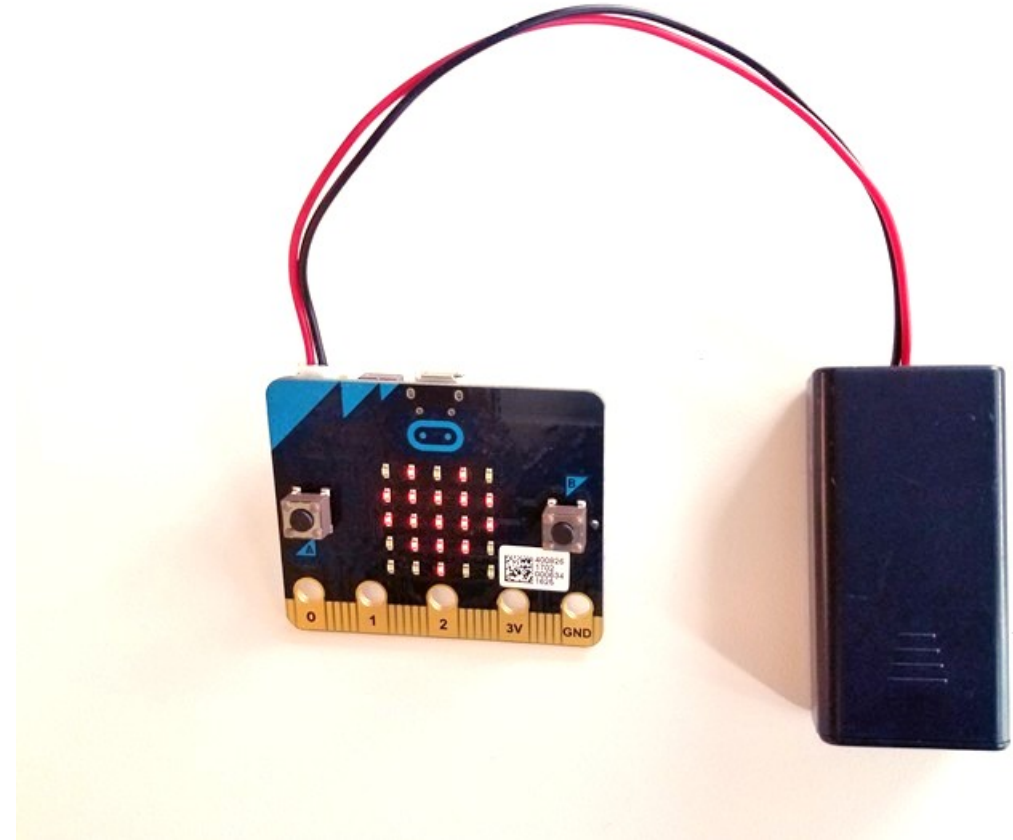
Advanced

Marco Polo mods

- Add a **‘show leds’** block to the **‘on start’** block to show an image of the initials MP.
- From the Music Toolbox drawer, drag two **‘play tone’** blocks to the coding workspace.
- Drag one of the **‘play tone’** blocks to the **‘on button A pressed’** block, and the other one to the **‘on button B pressed’** block.
- Change the default value in the **‘play tone’** block that is inside the **‘on button A pressed’** block to the value Low C.

Let's play Marco Polo!

1. Disconnect the micro:bit from your computer.
2. Attach the battery pack.
3. Only one person can be Polo – everyone presses Button A, except the one person who is Polo presses Button B.
4. Walk around and try to figure out who Polo is!



Morse code

- Morse Code uses dots, dashes, and stops (or spaces) to communicate.
- Turn your micro:bit into a Morse Code transmitter/receiver using the Radio.
- In our program:
 - The number 0 = dot
 - The number 1 = dash
 - The number 2 = stop/space



A • -	J • - - -	S • • •
B - • • •	K - • -	T -
C - • - •	L • - • •	U • • -
D - • •	M - -	V • • • -
E •	N - •	W • - -
F • • - •	O - - -	X - • • -
G - - •	P • - - •	Y - • - -
H • • • •	Q - - • -	Z - - • •
I • •	R • - •	

Pseudocode

- On Button A pressed, send 0
- On Button B pressed, send 1
- On Button A+B pressed, send 2
- When you receive a 0, show a dot
- When you receive a 1, show a dash
- When you receive a 2, show a stop/space

```

on button A pressed
  radio send number 0
  
```

```

on button B pressed
  radio send number 1
  
```

```

on button A+B pressed
  radio send number 2
  
```

```

on radio received receivedNumber
  if receivedNumber = 0 then
    show leds
    pause (ms) 1000
    clear screen
  else if receivedNumber = 1 then
    show leds
    pause (ms) 1000
    clear screen
  else
    show leds
    pause (ms) 1000
    clear screen
  
```

Let's play Morse code!

- In pairs, set a **unique Radio group number** to communicate just between you and your partner.
- Create a **secret message** for your partner **using Morse Code**.
- **Take turns** being the Sender and Receiver.
- Senders must transmit their message using the **micro:bit buttons**.
- Receivers must **de-code** the secret message!

on start

radio set group 47

Button A = dot

Button B = dash

Button A+B = stop/space

MORSE CODE

A • -

B - • • •

C - • - •

D - • •

E •

F • • - •

G - - •

H • • • •

I • •

J • - - -

K - • -

L • - • •

M - -

N - •

O - - -

P • - - •

Q - - • -

R • - •

S • • •

T -

U • • -

V • • • -

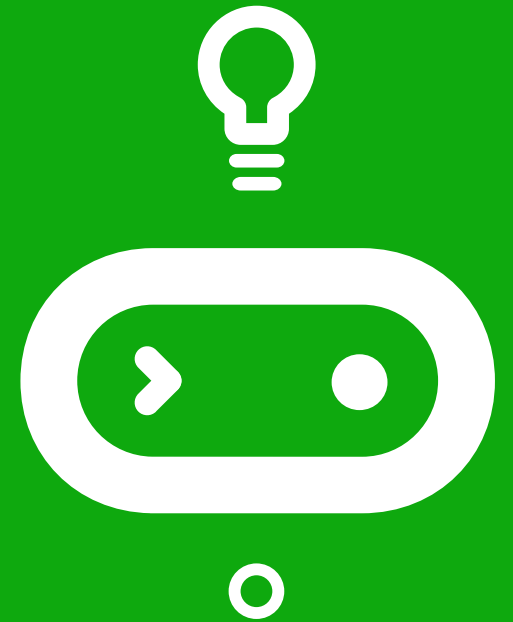
W • - -

X - • • -

Y - • - -

Z - - • •

Time for a quiz



What we learned in Lesson B

- Programmed the micro:bit to send and receive different types of data
- How much we've already learned



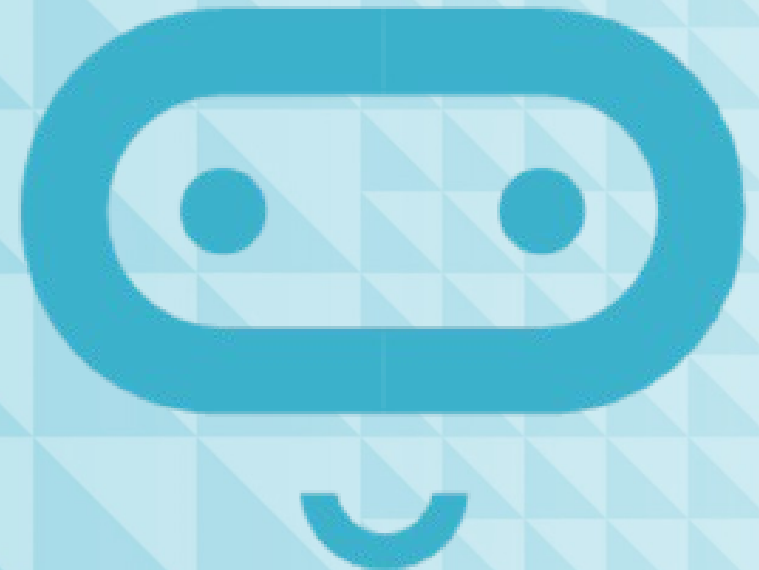
Next time: Lesson C

- Make a micro:bit radio



Lesson C

Make a micro:bit radio



What we'll learn in Lesson C

- Work in pairs to create a project using the micro:bit radio

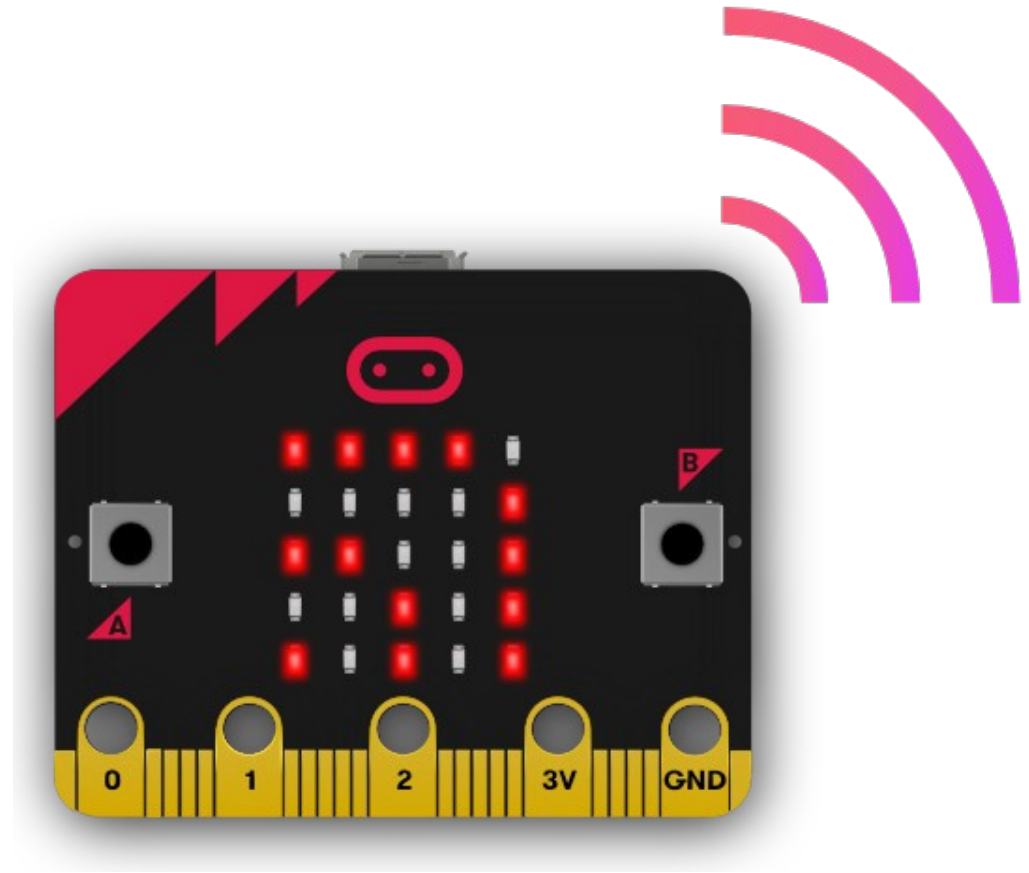
Pair Programming

- Computer programmers often collaborate on a project together. It's called “**pair programming**”:
- **Two people work together** on the same computer.
- One person is the **Navigator** – provides instructions and direction.
- One person is the **Driver** – at the keyboard/mouse taking instruction from the Navigator.
- Work together to ensure **good communication!**



Micro:bit radio

- Your task is to design a micro:bit project that incorporates radio communication to **send and receive data** in some way.
- You can create **2 separate programs** for the Sender and Receiver or use a **single program** that acts as both the Sender and Receiver.
- Use **strings**, **numbers**, or **string/number** value pairs.
- Your project should **solve a problem or fill a need**.



Project examples

Stop, thief!

Design an alarm system for your bedroom that alerts you with a screen animation when someone opens your door. You can mount one micro:bit on your door and use the accelerometer to send a signal over the radio when it is being moved.

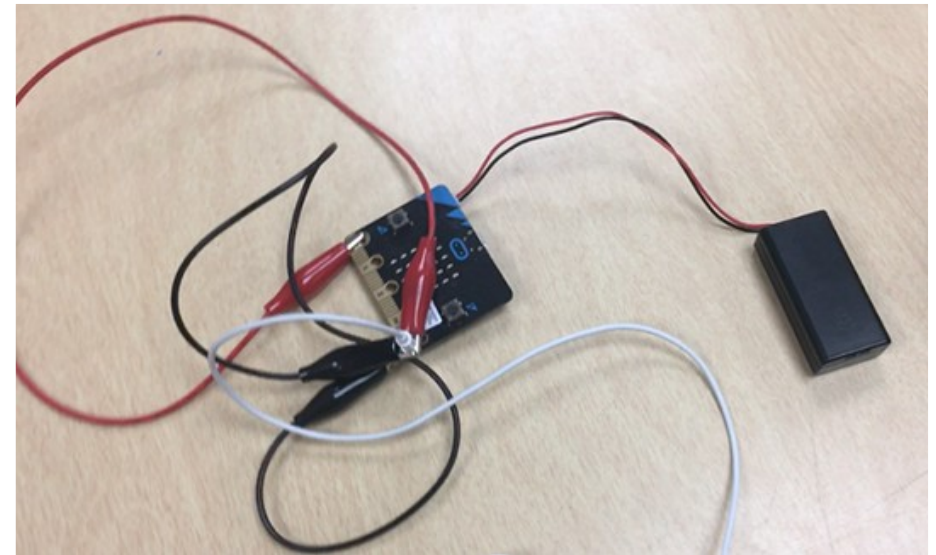
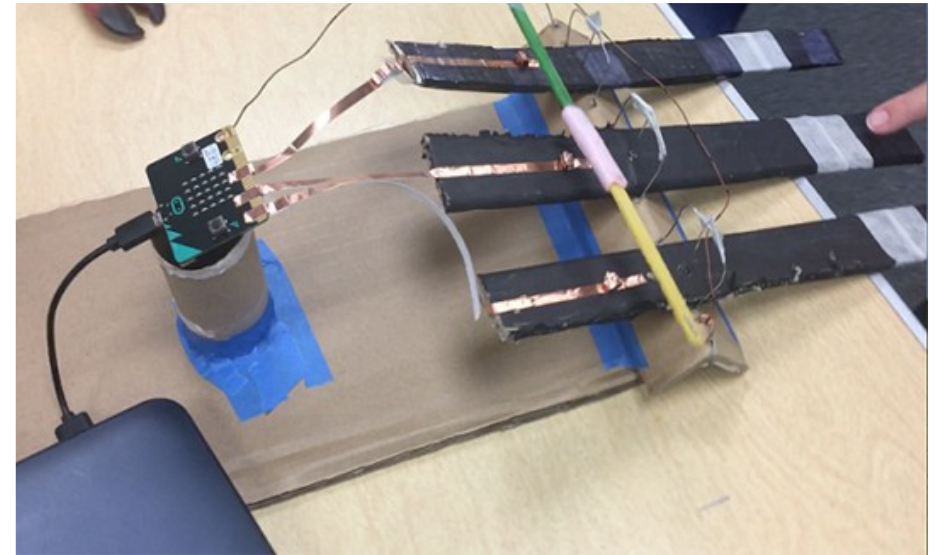
Interactive art

Create a piece of interactive artwork that receives something as input over the radio from another micro:bit and displays something based on that as output.



Three-note keyboard

- A simple three-note keyboard that uses wooden paint stirrers and copper tape to make a connection to the three micro:bit pins.
- When a key is pressed, it sends a number over the radio to a second micro:bit that plays the appropriate tone over a set of earbuds.
- This allows you to use each of the three pins on the first micro:bit to play a different tone.



Radio tennis

- The tennis racquets alternate displaying a ball on the micro:bit LED screen.
- When you swing the racquet, the ball disappears from one micro:bit display and shows up on the other micro:bit's display.



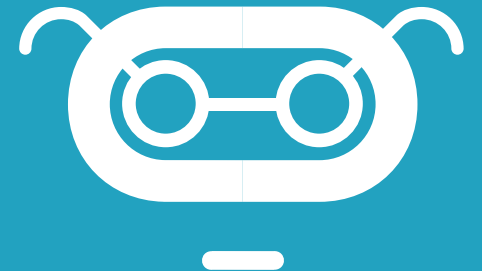
Project expectations

Use a design thinking approach and ensure the project meets these specifications:

- Uses the radio to send and receive data with **meaningful actions and responses** for each.
- Uses **Radio blocks** in a way that is **integral** to the program.
- The program **compiles and runs** as intended and includes **meaningful comments** in code.
- Provide the written **Reflection Diary** entry.

Reflection Diary

- What kind of project did you do? How did you decide what to pick?
- How does your project use radio communication?
- Are there separate programs for the Sender and the Receiver micro:bits? Or one program for both?
- Describe something in your project that you are proud of.
- Describe a difficult point in the process of designing this program and explain how you resolved it.
- What feedback did your testers give you? How did that help you improve your design?
- How would you improve your project, given more time?
- Relating to the pair programming process:
 - What challenges did you encounter working with a partner?
 - What benefits did you gain?
- Publish your MakeCode program and include the link.



What we learned in Lesson C

- Made a micro:bit radio

